Back-of-the-Envelope Estimation

the full URL shortener drill, in five minutes

Watch the 4-min video

In one line: state assumptions, then derive traffic, storage, bandwidth, cache and servers in order, rounding hard and tying each number to a design decision: for a URL shortener that is ~1k writes/s, ~100k reads/s, ~90 TB, ~20 GB of cache and ~100 servers.

1 The drill: five steps, out loud

In a real interview you get about **five minutes** for estimation, on a whiteboard, with someone watching. You need a fixed order so you never stall. This lesson runs the whole drill for a URL shortener, using every tool from this section: users, traffic, peak, storage, bandwidth, cache and servers.

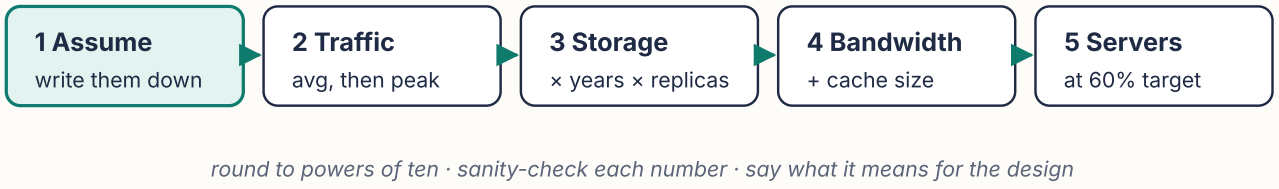


Figure 1. The order to work in. Each step feeds the next.

2 Step 1: state the assumptions, and write them down

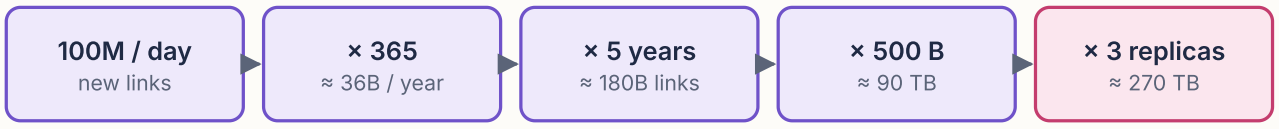
Assumption	Value
New short links per day	100 million
Read : write ratio	100 : 1
Retention	5 years
Size of each record	~500 bytes
Size of each redirect response	~500 bytes
Peak vs average	3×

Written assumptions let the interviewer correct you early, and let you redo the math fast if they change one.

3 Steps 2 and 3: traffic and storage

Traffic. A day has 86,400 seconds; round it to 10^5. Writes: 100M ÷ 10^5 = **~1,000 writes/s**. Reads are 100× that: **~100,000 redirects/s** on average. At 3× peak: **3,000 writes/s** and **300,000 reads/s**.

Storage. 100M links/day × 365 ≈ 36 billion a year, so about 180 billion over 5 years. Times 500 bytes is **~90 TB**. With 3 replicas, **~270 TB**.



270 TB is far beyond one server: a partitioned key-value store, keyed by short code

a 7-character base62 code gives $62^7 \approx 3.5$ trillion codes, plenty for 180 billion links

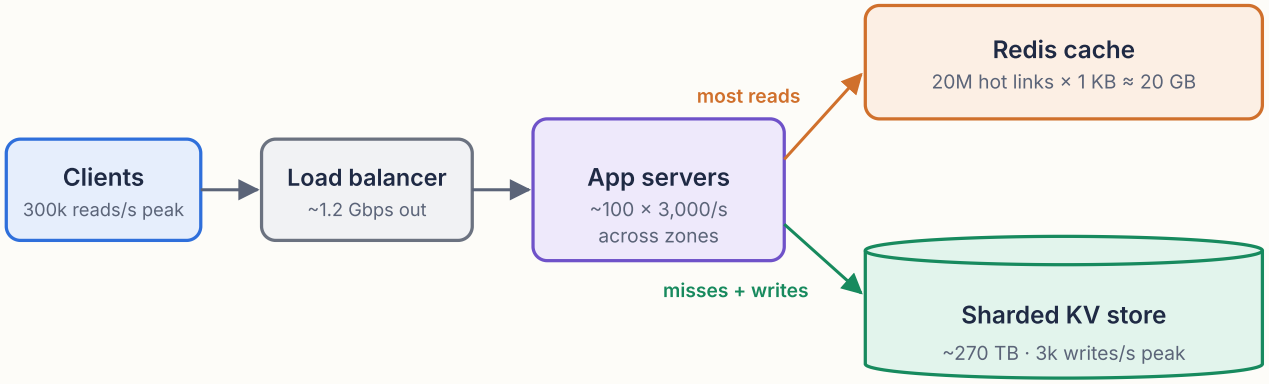
Figure 2. The storage chain. Keep one significant figure at each step; the answer only needs the right order of magnitude.

4 Steps 4 and 5: bandwidth, cache, servers

Bandwidth. Peak reads 300,000/s × 500 bytes = **~150 MB/s**, about **1.2 Gbps** of egress (× 8 bits per byte). Fine for a cluster.

Cache. Assume the daily set of distinct links read is about 100 million and 20% of them are hot: **20 million entries**. At about 1 KB each with key and overhead, that is **~20 GB of Redis**. Small enough for a few nodes.

Servers. If one app server handles 5,000 redirects/s and you target 60% utilization, plan for 3,000 each. 300,000 ÷ 3,000 = **~100 servers**, plus redundancy across zones. Most redirects hit the cache, so the database sees far fewer reads.



every box carries the number that justified it

Figure 3. The design the numbers point to: cache in front of a sharded key-value store, about 100 stateless app servers.

WORKED EXAMPLE: THE SUMMARY TABLE TO LEAVE ON THE BOARD

Quantity	Estimate	So the design needs...
Writes	~1k/s avg · 3k/s peak	modest; any KV store copes
Reads	~100k/s avg · 300k/s peak	a cache in front of storage
Storage (5 years)	~90 TB · 270 TB replicated	sharding / partitioned KV
Egress at peak	~150 MB/s ≈ 1.2 Gbps	nothing special
Cache	~20 GB	a few Redis nodes
App servers	~100 at 60%	stateless, spread across zones

5 How to do it well

- **Round aggressively.** Use powers of ten and keep one significant figure. 86,400 becomes 10^5; 36.5 billion becomes 36 billion.
- **Sanity-check every result.** Does 90 TB for five years of short links feel right? 180 billion rows of 500 bytes: yes.
- **Say what each number means.** "This needs sharding." "This fits in memory." A number with no consequence is wasted time.
- **Finish with a summary table** so the interviewer sees everything at a glance.

COMMON MISTAKES

- Starting to compute before stating assumptions.
- Sizing servers from the average instead of the peak.
- Forgetting replication (×3) in storage.
- Mixing bytes and bits: 150 MB/s is 1.2 Gbps, not 150 Mbps.
- Spending ten minutes on precise arithmetic the design does not need.

6 Key terms

Back-of-the-envelope: quick, rounded estimate to size a design

Peak factor: peak ÷ average traffic; here 3×

Hot set: the share of data most requests touch

Read : write ratio: reads per write; here 100 : 1

Egress: data sent out to clients

Target utilization: planned load per server, e.g. 60%

7 Interview questions

Q1. 100 million new links a day. Writes per second? BASIC

100M ÷ 10^5 seconds ≈ 1,000 writes/s on average, about 3,000 at a 3× peak. (Exact: 100M ÷ 86,400 ≈ 1,160.)

Q2. How much storage for 5 years at 500 bytes per link? BASIC

100M × 365 × 5 ≈ 180 billion links × 500 B ≈ 90 TB, or about 270 TB with 3 replicas.

Q3. Peak reads are 300k/s at 500 bytes. What bandwidth? BASIC

300,000 × 500 B = 150 MB/s, × 8 = 1.2 Gbps of egress.

Q4. How big is the cache? SCENARIO

If 20% of a 100-million daily link set is hot, that's 20 million entries at about 1 KB each: ~20 GB, a few Redis nodes.

Q5. How many app servers? SCENARIO

One server does 5,000 redirects/s; at a 60% target, 3,000. 300,000 ÷ 3,000 = 100 servers, spread across zones for redundancy.

Q6. The interviewer says reads are 1,000 : 1, not 100 : 1. What changes? SCENARIO

Reads become ~1M/s average and 3M/s peak; egress ~12 Gbps; about 1,000 app servers. Storage and writes do not change. The cache hit rate and a CDN now matter much more.

Q7. Why is the database not the bottleneck at 300k reads/s? TRICKY

The cache absorbs most redirects. At a 90% hit rate only ~30k reads/s reach storage, spread over many shards. The database's real problem is its size, 270 TB.

Q8. Why round 86,400 to 100,000? TRICKY

It makes mental math instant and errs about 15% low, which is well inside the error of the assumptions. Say that you rounded, and keep going.

Next: SD-D-012 · Architecture Diagrams backendroadmap.com